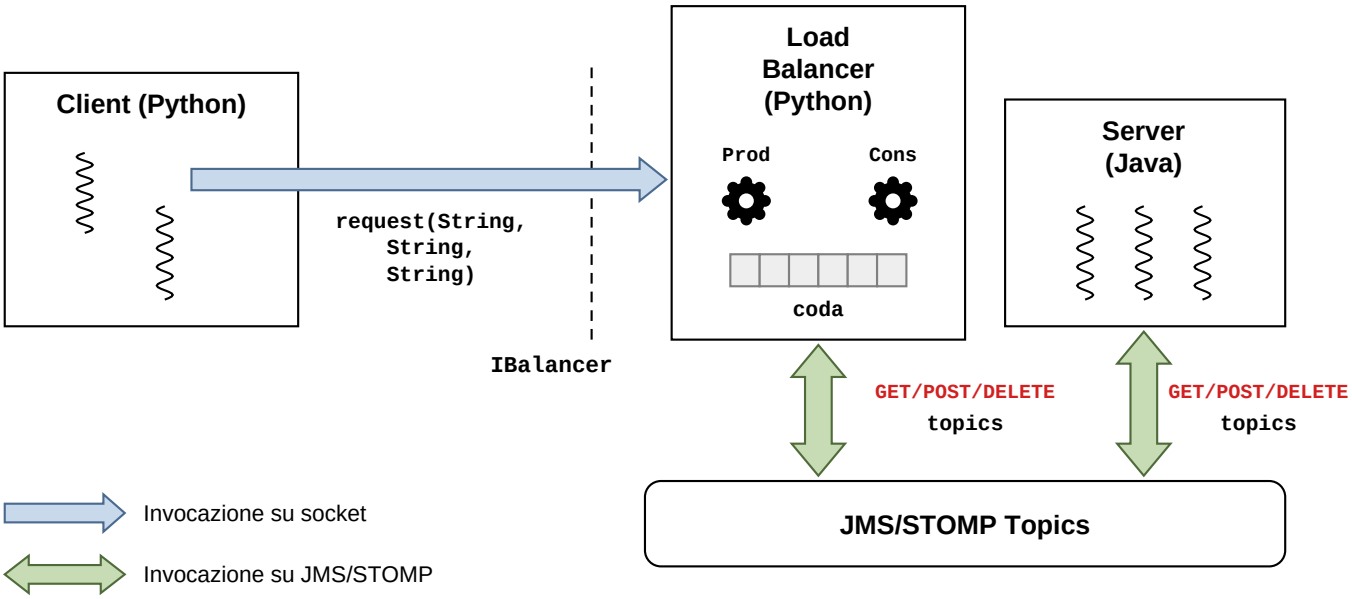
Prova pratica del giorno 20/07/2026
Durata della prova: 120 minuti

Testo della prova



Il candidato implementi un sistema distribuito in **Python** e **Java** per la gestione di richieste HTTP basato su **Socket TCP** e **JMS**. Il sistema è caratterizzato dai seguenti componenti.

Client. E' una applicazione Python multiprocesso che genera le richieste da inviare al **Load Balancer**. L'invio di una richiesta consiste nell'invocazione del metodo ***int request(String, String, String)*** specificato nell'interfaccia **IBalancer**. La richiesta è caratterizzata da 1) **tipo metodo HTTP** (*String*), ossia la tipologia di sensore (GET, POST, DELETE); 2) **domain** (*String*) (e.g., api.example.com) 3) **endpoint** (*String*), ossia l'endpoint specificato nella richiesta HTTP (e.g., /posts/123). Il **Client** crea 5 thread clienti, che invocano il metodo *request* per 5 volte (attendendo 1 secondo tra le invocazioni). Le richieste possono essere statiche o generate randomicamente.

Load Balancer. Componente Python che fornisce l'interfaccia *IBalancer* e il relativo metodo *int request(String, String, String)*. All'invocazione del metodo *request*, si avvia un processo produttore, il quale inserisce in una **coda di dimensione *N* = 6** (*process-safe* e che implementi il problema del produttore/consumatore) una stringa che concateni tutti i parametri della richiesta ottenendo ogni elemento del tipo `POST https://api.example.com/posts/123`).
I dati inseriti nella coda sono consumati da un processo consumatore avviato al lancio del ***Load Balancer***. Il processo consumatore, per ogni richiesta consumata, funge da publisher che invia un opportuno messaggio STOMP su un topic specifico (*GET, POST, DELETE*) a seconda della tipologia del metodo HTTP della richiesta consumata.

Server. E' una applicazione **Java multithread** che crea 3 thread, uno per ogni tipologia di metodo HTTP. Ogni thread crea una subscription durabile sul topic opportuno (e.g., thread GET farà la subscription al topic *GET*). La ricezione dei messaggi deve essere asincrona attraverso un opportuno *listener* distinto, uno per ogni topic. Ogni listener, alla ricezione del messaggio lo stamperà a video.

Il candidato utilizzi proxy-skeleton con socket TCP per la comunicazione tra Client e Load Balancer, e JMS/STOMP per quella tra Load Balancer e Server. A tal fine, il candidato predisponga le opportune interfacce e le classi Proxy-Skeleton. Si utilizzi inoltre skeleton per delega per il Load Balancer.